

CE392: Sliding-window NPU / Reconfigurable CNN Accelerator

1 Introduction

1.1 Motivation

Convolutional neural networks (CNNs) are widely used in edge AI applications, and face verification is a representative workload in this domain. Compared with simple classification tasks, practical face verification systems require repeated convolution over spatially correlated image regions, feature extraction across multiple layers, and efficient support for lightweight embedding networks. Therefore, an accelerator for this application should not only provide sufficient compute capability, but also support efficient data reuse and flexible layer execution.

Our previous work focused on a fixed ASIC MNIST NPU. That design used a fixed 4-layer network for digit classification and a 32-lane time-multiplexed architecture with resource sharing. While effective for a small and fixed workload, it has two main limitations. First, the supported layer sizes and dataflow were largely fixed, so the design was not suitable for other CNN models. Second, the architecture did not explicitly exploit sliding-window based spatial reuse, which limits efficiency for practical convolution-heavy workloads.

For this reason, this project aims to build a **configurable sliding-window CNN accelerator** for face verification. The main motivation is threefold:

- CNN-based applications require efficient convolution support.
- Sliding-window buffering can exploit spatial data reuse.
- A configurable architecture better matches practical AI workloads than a fixed small network.

As shown in Figure 1, the previous ASIC MNIST NPU was implemented for a fixed small-network workload. This figure provides the physical layout view of that design and serves as a reference point for the motivation of the proposed configurable accelerator.

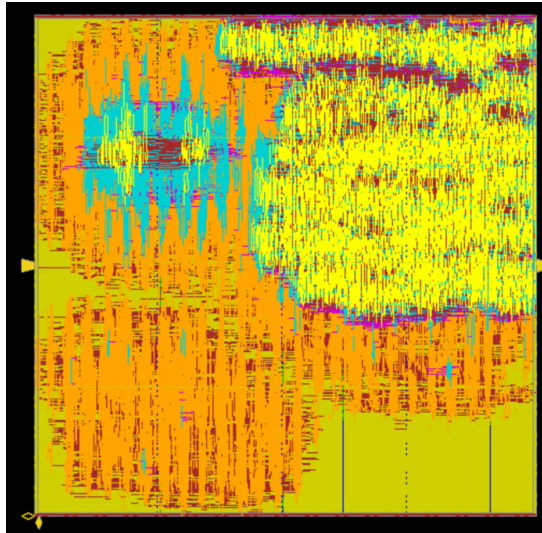


Figure 1: Layout of the previous ASIC MNIST NPU design

Table 1: Comparison of candidate backbone networks.

Feature	MobileNetV2	MobileFaceNet	EdgeFace
Primary Use	General Object Classification	Lightweight face embedding network	Lightweight face recognition / verification for edge devices
Architecture	Inverted Residuals (CNN)	CNN + GDC	Hybrid (CNN-Transformer)
Parameters	~3.4M	< 1M	~1.5M - 2M (Scaled versions)
Strength	Strong general mobile baseline	Very efficient for face embeddings	Stronger recent accuracy (Handles pose/occlusion better)
Hardware Friendliness	High	High	Medium

1.2 Key Features of the Project

Based on the above motivation, this project proposes a configurable CNN accelerator targeting lightweight face verification workloads. The key features of the project are summarized as follows:

- **Configurable layer execution.** Instead of hard-coding one fixed network, the accelerator accepts layer-level configuration such as feature-map size, channel count, kernel size, stride, and activation type.
- **Sliding-window based data reuse.** The architecture uses line buffers and sliding-window generation to reduce redundant input reads during convolution.
- **Reusable compute core.** A shared PE/MAC array is used across multiple CNN layers, improving hardware utilization.
- **Application-oriented design.** The target application is face verification, and the initial mapping target is a lightweight face embedding model.
- **FPGA-oriented prototype flow.** The design will be implemented and verified in RTL, then prototyped on FPGA for performance evaluation.

These features define the main contributions of this project: introducing configurability into a lightweight CNN accelerator, combining it with sliding-window data reuse, and evaluating the design under edge-oriented metrics such as throughput, power, and area.

2 Architecture and Algorithm

2.1 Target Application and Backbone Selection

The final target application of this project is **face verification**. In a typical face verification pipeline, the input face image is first aligned and then processed by a lightweight backbone network to generate a compact embedding vector. Verification is then performed by comparing embedding similarity. From a hardware perspective, the backbone network is the dominant workload, so the accelerator design should primarily optimize its computation.

In this project, three candidate lightweight backbones are considered: MobileNetV2, MobileFaceNet, and EdgeFace. Their features are summarized in Table 1. This table should be kept in the report because it directly explains why different models are considered and why one of them is chosen as the primary implementation target.

From the implementation point of view, **MobileFaceNet** is selected as the primary target of this project. The reason is that it is more NPU-friendly: its structure remains predominantly CNN-based, its parameter count is small, and its operators are easier to map onto a reusable PE/MAC array with line-buffer based convolution support. In contrast, EdgeFace is a more recent hybrid CNN-Transformer model and is a meaningful future research direction. After a configurable NPU baseline is established, EdgeFace can be studied later as a comparison target against MobileFaceNet in terms of accuracy and hardware cost.

2.2 MobileFaceNet Architecture

MobileFaceNet is a lightweight face embedding network designed specifically for face verification. Its structure contains an initial convolution layer, several bottleneck blocks, a 1×1 projection layer, a global depthwise convolution layer, and a final linear projection to the embedding dimension. Compared with general-purpose lightweight CNNs, MobileFaceNet is better aligned with the face verification task and offers a better balance between accuracy and efficiency.

Figure 2 should be used here to show the overall architecture of MobileFaceNet. This figure directly corresponds to the description in this subsection and helps the reader understand the workload that will be mapped onto the accelerator.

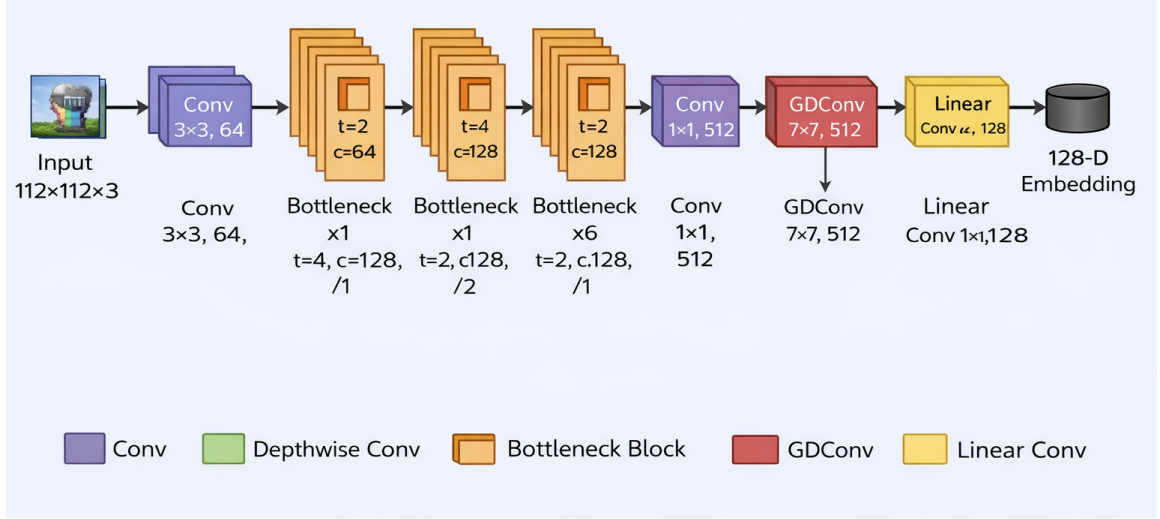


Figure 2: MobileFaceNet architecture used as the primary workload target.

In this project, MobileFaceNet is not directly hard-coded into the hardware. Instead, its layers are represented as a sequence of configurations, and the NPU executes them layer by layer. This is the key difference between the proposed design and a fixed accelerator.

2.3 Configurable NPU Architecture

The proposed hardware is a **configurable sliding-window CNN accelerator**. Its main idea is to separate the CNN model description from the hardware datapath. Instead of fixing all layer parameters in RTL, the architecture uses layer configuration words to describe feature-map size, channel count, kernel size, stride, and activation function. A programmable controller then decodes this information and schedules the corresponding layer execution.

Figure 3 should be used first to present the high-level execution flow of the design. It should show that layer configuration is interpreted by the controller, which then drives the input/weight buffers, sliding-window unit, PE/MAC array, and output buffer.

At the system level, the architecture contains the following main modules:

- input interface,
- external memory / DRAM interface,
- input buffer and line buffer,
- weight buffer,
- scheduler / controller,
- PE / MAC array,

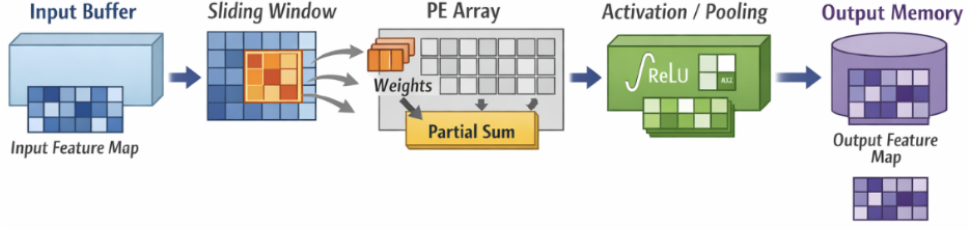


Figure 3: High-level execution flow of the proposed configurable CNN accelerator.

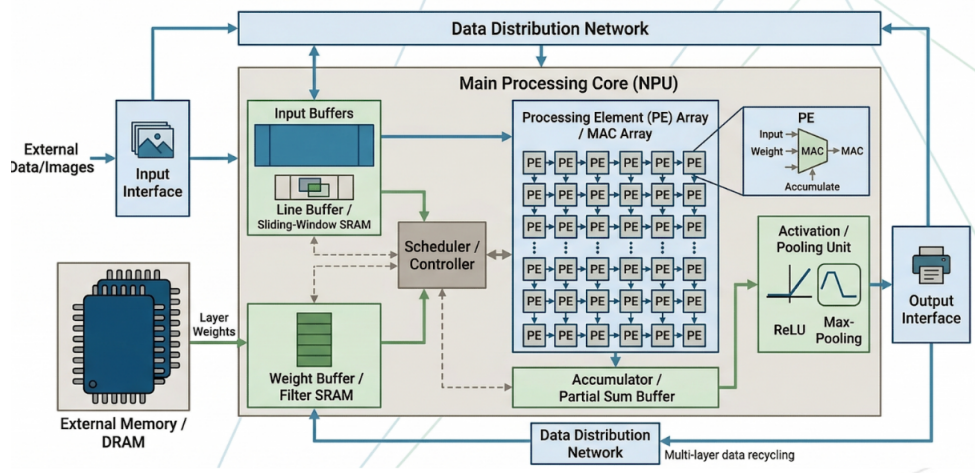


Figure 4: System architecture of the proposed configurable NPU.

- accumulator / psum buffer,
- activation / pooling unit,
- output interface.

Figure 4 should then be used to illustrate the detailed system architecture. This figure is connected to the module list above and provides the hardware view of the design.

This subsection is intentionally kept brief, because the detailed hardware block design will be further developed by the team in the implementation stage.

3 Simulation and Verification Methodology

This project evaluates the proposed design using four metrics: **TOPS**, **throughput**, **power**, and **area**. Since the target application is edge AI, power and area are considered together with performance.

3.1 Evaluation Metrics

- **TOPS:** peak arithmetic capability of the accelerator.
- **Throughput:** effective processing rate under the target clock frequency.
- **Power:** estimated hardware power for edge-oriented deployment.
- **Area:** hardware cost, mainly reflected by synthesis or FPGA resource usage.

3.2 Verification Flow

The verification flow contains three steps.

First, a simple software golden model will be used to generate reference results for selected convolution layers and simplified MobileFaceNet-style workloads.

Second, the RTL design will be verified at the module level, including the line buffer, PE/MAC array, controller, and partial-sum path. After that, the main modules will be integrated for system-level simulation.

Third, the design will be prototyped on an FPGA. FPGA results will be used to check execution correctness and collect performance-related data such as frequency, throughput, resource usage, and tool-reported power estimation.

3.3 Expected Results

The purpose of this methodology is to verify two aspects: correctness and efficiency. Correctness is checked by comparing RTL outputs with the software reference. Efficiency is evaluated by FPGA-based measurements and synthesis results using the four metrics listed above.

4 Development Plan

4.1 Plan Flow

The project development plan is organized according to the course schedule and divided into six stages.

- **Week 1–2: Background study**
Literature review and workload analysis.
- **Week 3–4: Architecture design**
Define the configurable NPU architecture, target workload, and verification flow. This is the current stage of the project.
- **Week 4–6: RTL implementation**
Complete RTL design of the main hardware blocks, including line buffer, weight buffer, PE/MAC array, controller, and psum buffer.
- **Week 8: Verification**
Perform module integration, simulation, and comparison with the software golden model.
- **Week 9: Backend design**
Conduct synthesis, resource analysis, timing evaluation, and FPGA implementation preparation.
- **Week 10–11: Final report and demo**
Complete final documentation, summarize experimental results, and prepare the final demonstration.

4.2 Task Allocation

The project is carried out by two team members.

Peixin Zhang will mainly focus on architecture and hardware implementation. The tasks include workload analysis, configurable datapath definition, RTL implementation of the line buffer, controller, and PE/MAC array, as well as FPGA integration.

Willie Liao will mainly focus on verification and evaluation support. The tasks include software golden model development, testbench construction, correctness checking, performance data collection, and assistance with synthesis and backend analysis.

Some tasks will be completed collaboratively, especially system integration, result interpretation, and final report writing.

5 Final Deliverables

The final deliverable of this project is a **configurable sliding-window CNN accelerator prototype** for lightweight face verification workloads.

The expected outputs of the project are listed below:

- A configurable accelerator architecture description, including the main modules and layer configuration flow.
- RTL implementation of the key hardware blocks, including buffers, controller, PE/MAC array, and output accumulation path.
- A basic software reference model for selected layer-level correctness checking.
- Simulation results showing functional correctness of the implemented design.
- An FPGA prototype of the integrated design, together with frequency, throughput, resource usage, and power estimation results.
- A final report and demo presentation summarizing the design, implementation, and evaluation results.

Overall, the goal of the project is to deliver a working configurable CNN accelerator prototype and demonstrate its feasibility for lightweight face verification workloads.

References

- [1] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. DOI: 10.1109/JSSC.2016.2616357.
- [2] A. George, C. Ecabert, H. O. Shahreza, K. Kotwal, and S. Marcel, “Edgeface: Efficient face recognition model for edge devices,” *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 6, no. 2, pp. 158–168, 2024. DOI: 10.1109/TBIOM.2024.3352164.
- [3] S. Chen, Y. Liu, X. Gao, and Z. Han, *Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices*, 2018. arXiv: 1804.07573 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1804.07573>.